

Ali Moallin

Electrical and Computer Engineering

Regarding how I wrote the VHDL code for the System Controller state diagram [4], I made it so the implementation was a finite state machine with 5 states. These states being InitS, LoadS, AddS, ShiftS, and DoneS. The controller updates on the falling edge and the registers update on the rising edge.

The original controller template included a counter (CNT), I moved this counter's behavior into its own VHDL module, naming it CounterN [5]. I made the counter have ports, Clk, Clear, Enable, and C4. Clear is connected to Load, Enable is connected to Shift, and C4 is connected into the controller.

The waveform [1] shows that the multiplier works because when Start is asserted, the circuit starts the multiplication cycle, after the cycles, Done becomes high, this shows the operation is finished. Also, the product signal matches the multiplication result for corresponding Multiplier and Multiplicant values.

One modification that I think would be very convenient is a RESET, this makes it so the user does not have to wait for several cycles, but manually reset it whenever they want. This can be very useful in certain situations.

Appendix

Figure 1 Wave Simulation

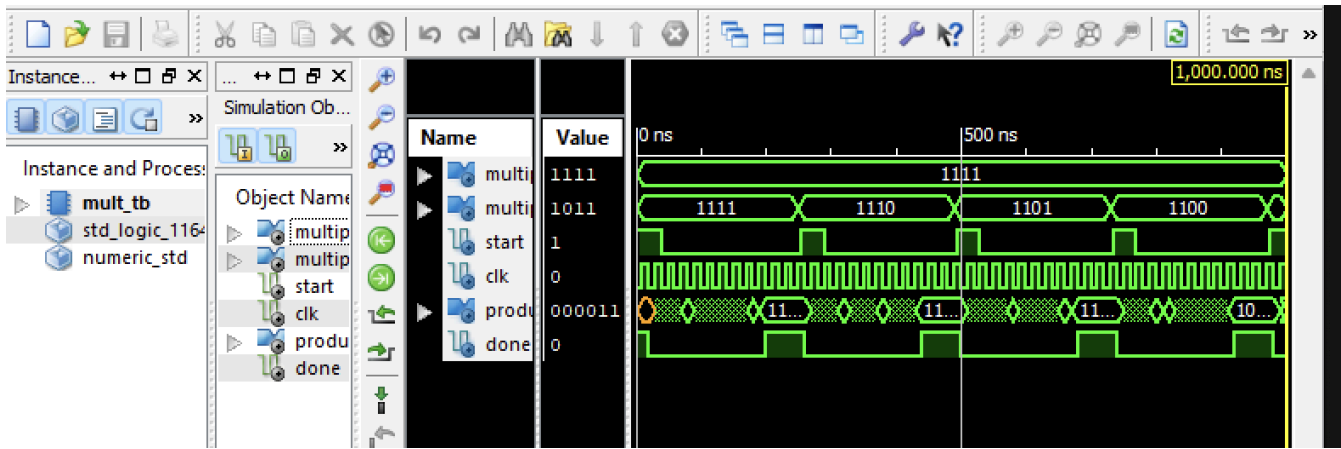


Figure 2 Test Bench

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY mult_tb IS
6  END mult_tb;
7
8  ARCHITECTURE behavior OF mult_tb IS
9
10     COMPONENT MULTTOP
11     PORT(
12         Multiplier : IN  std_logic_vector(3 downto 0);
13         Multiplicand : IN  std_logic_vector(3 downto 0);
14         Product : OUT  std_logic_vector(7 downto 0);
15         Start : IN  std_logic;
16         Done : OUT  std_logic;
17         Clk : IN  std_logic
18     );
19     END COMPONENT;
20
21     --Inputs
22     signal Multiplier : std_logic_vector(3 downto 0) := (others => '0');
23     signal Multiplicand : std_logic_vector(3 downto 0) := (others => '0');
24     signal Start : std_logic := '0';
25     signal Clk : std_logic := '0';
26
27     --Outputs
28     signal Product : std_logic_vector(7 downto 0);
29     signal Done : std_logic;
30
31 BEGIN
32
33     -- Instantiate the Unit Under Test (UUT)
34     uut: MULTTOP PORT MAP (
35         Multiplier => Multiplier,
36         Multiplicand => Multiplicand,
37         Product => Product,
38         Start => Start,
39         Done => Done,
40         Clk => Clk
41     );
42
43     Clk <= not Clk after 10 ns;
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

Figure 3 *Multiplier*

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity MULTTOP is
5      Port ( Multiplier : in  STD_LOGIC_VECTOR (3 downto 0);
6            Multiplicand : in  STD_LOGIC_VECTOR (3 downto 0);
7            Product : out  STD_LOGIC_VECTOR (7 downto 0);
8            Start : in  STD_LOGIC;
9            Done : out  STD_LOGIC;
10           Clk : in  STD_LOGIC);
11 end MULTTOP;
12
13 architecture Behavioral of MULTTOP is
14     component Controller
15         generic (N: integer := 2);
16         Port ( Clk : in  STD_LOGIC;
17               Q0 : in  STD_LOGIC;
18               C4 : in  STD_LOGIC;
19               Start : in  STD_LOGIC;
20               Load : out  STD_LOGIC;
21               Shift : out  STD_LOGIC;
22               AddA : out  STD_LOGIC;
23               Done : out  STD_LOGIC);
24     end component;
25
26     component AdderN
27         generic (N: integer := 4);
28         port ( A: in std_logic_vector(N-1 downto 0);
29               B: in std_logic_vector(N-1 downto 0);
30               S: out std_logic_vector(N downto 0)
31             );
32     end component;
33
34     component RegN
35         generic (N: integer := 4);
36         port ( Din: in std_logic_vector(N-1 downto 0);
37               Dout: out std_logic_vector(N-1 downto 0);
38               Clk: in std_logic;
39               Load: in std_logic;
40               Shift: in std_logic;
41               Clear: in std_logic;
42               SerIn: in std_logic
43             );
44     end component;
```

Figure 4 Controller

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity Controller is
6      generic (N: integer := 2);
7      Port ( Clk : in  STD_LOGIC;
8            Q0 : in  STD_LOGIC;
9            C4 : in  STD_LOGIC;
10           Start : in  STD_LOGIC;
11           Load : out STD_LOGIC;
12           Shift : out STD_LOGIC;
13           AddA : out STD_LOGIC;
14           Done : out STD_LOGIC);
15 end Controller;
16
17 architecture Behavioral of Controller is
18     type states is (
19
20 -- Uncomment the following library declaration if instantiating
21 -- any Xilinx primitives in this code.
22 --library UNISIM;
23 --use UNISIM.VComponents.all;
24 InitS,LoadS,AddS,ShiftS, DoneS);
25     signal state:states := InitS;
26
27     begin
28     Done <= '1' when state = InitS or state = DoneS else '0';
29     Load <= '1' when state = LoadS else '0';
30     AddA <= '1' when state = AddS else '0';
31     Shift <= '1' when state = ShiftS else '0';
32     process(Clk)
33     begin
34         if falling_edge(Clk) then
35             case state is
36
37                 --InitS waits for Start
38
39                 when InitS =>
40                     if Start = '1' then
41                         state <= LoadS;
42                     else
43                         state <= InitS;
44                     end if;

```

```

44         end if;
45
46         -- InitS loads the multiplier and multiplicand regs
47         --controller checks current multiplier
48         when LoadS =>
49             if Q0 = '1' then
50                 state <= AddS;
51             else
52                 state <= ShiftS;
53             end if;
54
55         --Adds loads adder result into accumulator
56
57         when AddS =>
58             state <= ShiftS;
59
60         --shiftS shifts the accumulator and multipler regs
61         -- ifC4 is high, the multiplier bits are processed
62         when shiftS =>
63             if C4 = '1' then
64                 state <= DoneS;
65             elsif Q0 = '1' then
66                 state <= AddS;
67             else
68                 state <= ShiftS;
69
70             end if;
71
72         --DoneS holds done high until start goes low
73         when DoneS =>
74             if Start = '0' then
75                 state <= InitS;
76             else
77                 state <= DoneS;
78             end if;
79
80
81         | case;
82         ;
83         ;
84         ;
85         ;
86         ;
87         ;

```

Figure 5 Counter

```
1
2 library IEEE;
3 use IEEE.STD_LOGIC_1164.ALL;
4 use IEEE.NUMERIC_STD.ALL;
5
6
7
8 entity CounterN is
9     generic (N : integer := 3);
10    port ( Clk : in  STD_LOGIC;
11          Clear : in  STD_LOGIC;
12          Enable : in  STD_LOGIC;
13          C4 : out  STD_LOGIC);
14 end CounterN;
15
16 architecture Behavioral of CounterN is
17
18    --internal counter sig
19    --N=3 gives a 3-bit counter, can count up to 4
20    signal Count : unsigned(N-1 downto 0) := (others => '0');
21
22
23 begin
24
25    process(Clk)
26    begin
27        if rising_edge(Clk) then
28
29            -- clear the counter at start of multiplication
30            if Clear = '1' then
31                Count <= (others => '0');
32
33            elsif Enable = '1' then
34                Count <= Count +1;
35            end if;
36
37        end if;
38
39    end process;
40
41    -- c4 becomes high afyer 4 shift cycles
42    C4<= '1' when Count = to_unsigned(4,N) else '0';
43
44 end Behavioral;
```

Figure 6 Adder

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.NUMERIC_STD.ALL;
4
5 entity AdderN is
6     generic (N: integer := 4);
7     port ( A: in std_logic_vector(N-1 downto 0);
8           B: in std_logic_vector(N-1 downto 0);
9           S: out std_logic_vector(N downto 0)
10    );
11 end AdderN;
12
13 architecture Behavioral of AdderN is
14 begin
15     S <= std_logic_vector(('0' & UNSIGNED(A)) + UNSIGNED(B));
16 end Behavioral;
17
```

Figure 7 *Register*

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity RegN is
5      generic (N: integer := 4);
6      port ( Din: in std_logic_vector(N-1 downto 0);
7            Dout: out std_logic_vector(N-1 downto 0);
8            Clk: in std_logic;
9            Load: in std_logic;
10           Shift: in std_logic;
11           Clear: in std_logic;
12           SerIn: in std_logic
13       );
14  end RegN;
15
16  architecture Behavioral of RegN is
17      signal Dinternal: std_logic_vector(N-1 downto 0);
18  begin
19      process(Clk)
20      begin
21          if (rising_edge(Clk)) then
22              if (Clear = '1') then
23                  Dinternal <= (others => '0');
24              elsif (Load = '1') then
25                  Dinternal <= Din;
26              elsif (Shift = '1') then
27                  Dinternal <= SerIn & Dinternal(N-1 downto 1);
28              end if;
29          end if;
30      end process;
31      Dout <= Dinternal;
32  end Behavioral;
33

```

The system controller-based architecture is shown in Figure 1. There are two 4-bit shift registers (74LS194), a custom 9-bit shift register, a 4-bit counter (74LS163), a 4-bit parallel adder, and a system controller. The custom register operates exactly like 74LS194 except it holds 9 bits instead of 4 bits. Their control inputs S_1 and S_0 operate as: $S_1S_0 = \{00, 01, 10, 11\} = \{Hold, Shift Right, Shift Left, Load\}$. The 4-bit parallel adder takes two 4-bit numbers ($X_4X_3X_2X_1$) and ($Y_4Y_3Y_2Y_1$) along with an initial carry C_1 and produces their sum ($S_4S_3S_2S_1$) and a new carry C_5 .

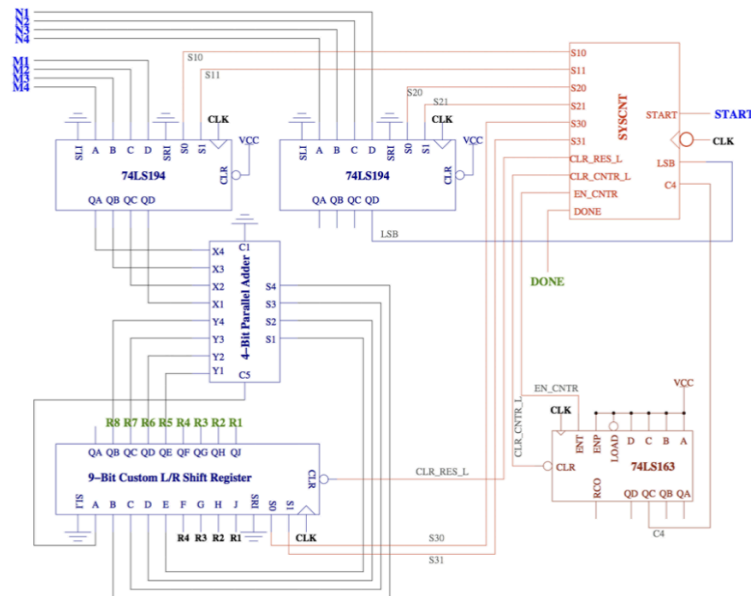


Figure 1: System controller-based architecture for 4-bit binary multiplier showing three shift registers, one counter, one parallel adder, and one system controller.

Multiplicand $\rightarrow$ 1101 (13)
 Multiplier $\rightarrow$ 1011 (11)
 Partial Product $\rightarrow$ 1101
 Partial Product $\rightarrow$ 1101
 Partial Product $\rightarrow$ 0000
 Partial Product $\rightarrow$ 1101
 Product $\rightarrow$ 10001111 (143)

Figure 2: Overall binary multiplication process.

initial contents of product register 0 0 0 0 0 1 0 1 1 $\leftarrow M$ (11)
 (add multiplicand because $M = 1$) 1 1 0 1 (13)
 after addition 0 1 1 0 1 1 0 1 1
 after shift 0 0 1 1 0 1 1 0 1 $\leftarrow M$
 (add multiplicand because $M = 1$) 1 1 0 1
 after addition 1 0 0 1 1 1 1 0 1
 after shift 0 1 0 0 1 1 1 1 0 $\leftarrow M$
 (skip addition because $M = 0$)
 after shift 0 0 1 0 0 1 1 1 1 $\leftarrow M$
 (add multiplicand because $M = 1$) 1 1 0 1
 after addition 1 0 0 0 1 1 1 1 1
 after shift (final answer) 0 1 0 0 0 1 1 1 1 (143)
 dividing line between product and multiplier

Figure 3: Detailed binary multiplication process showing contents of each shift register in the system controller-based architecture.

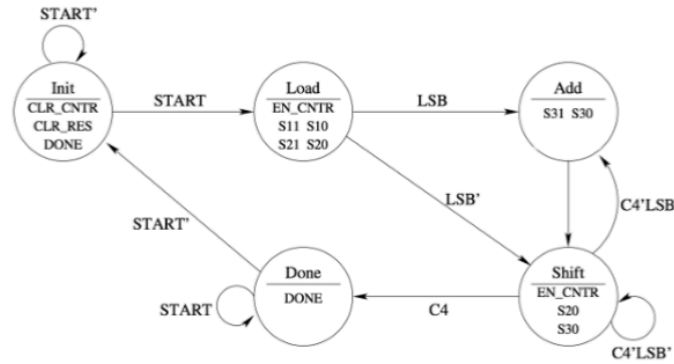


Figure 4: State diagram for the system controller in the 4-bit binary multiplier circuit.